Homework 1

AI, Neural Networks, Large Language Models, and Agentic AI Systems

Student: Ara Davtyan
Prepared: 13 October 2026

Tasks 1–4 contain direct constructions and proofs. Task 5 contains real Silverbox experiments. Task 6 infrastructure is implemented and tested; its real workflow has not completed successfully. The Task 6 section records the actual latest run status. GitHub publication remains pending.

Task 1. ReLU networks

Write $R(z)=\max(0,z)$. Hidden neurons use R; every output below uses the identity activation. No skip connections are used.

(a) Absolute value. The two hidden neurons have weights 1 and -1, and biases 0 and 0. The output weights are (1,1), and the output bias is 0:

$$h_1=R(x),\ h_2=R(-x),\ f(x)=h_1+h_2=|x|.$$

For $x\geq 0$ the hidden values are $(x,0)$; for $x\leq 0$ they are $(0,-x)$. Their sum is $|x|$ on the entire real line, including zero. Two hidden units are minimal: with one hidden ReLU, $A\,R(wx+b)+c$ is constant on one unbounded half-line (or everywhere if $w=0$), whereas $|x|$ is nonconstant on both tails.

x	Hidden pre-activations	Hidden activations	Output
-2	(−2, 2)	(0, 2)	2
-0.5	(−0.5, 0.5)	(0, 0.5)	0.5
0	(0, 0)	(0, 0)	0
0.5	(0.5, −0.5)	(0.5, 0)	0.5
3	(3, −3)	(3, 0)	3

(b) XOR. Hidden weights are $(1,-1)$ and $(-1,1)$, both biases 0. Output weights are (1,1), bias 0, with identity activation:

$$h_1=R(x_1-x_2),\ h_2=R(x_2-x_1),\ f=h_1+h_2=|x_1-x_2|.$$

Input	(h_1,h_2)	Output
(0,0)	(0,0)	0
(0,1)	(0,1)	1
(1,0)	(1,0)	1
(1,1)	(0,0)	0

Two hidden units are minimal: thresholding a one-ReLU affine-output network at 1/2 gives a half-space, but XOR labels opposite corners alike, and the two diagonal pairs have the same midpoint, so no affine separator exists. On Boolean inputs, absolute difference is exactly XOR.

Task 2. Parameter counting

For each spatial layer, $H_{out}=\lfloor(H+2p-k)/s\rfloor+1$ and likewise for width. A convolution has $k^2C_{in}C_{out}$ weights and C_{out} biases. A dense layer has $N_{in}N_{out}$ weights and N_{out} biases. ReLU and max-pooling each contribute zero parameters.

Layer	Spatial calculation	Output shape
C1	(224+6−7)//2+1=112	112 × 112 × 64
P1	(112−3)//2+1=55	55 × 55 × 64
C2	(55+4−5)//1+1=55	55 × 55 × 192
P2	(55−3)//2+1=27	27 × 27 × 192
C3	(27+2−3)//1+1=27	27 × 27 × 384
C4	(27+2−3)//1+1=27	27 × 27 × 256
C5	(27+2−3)//1+1=27	27 × 27 × 256
P3	(27−3)//2+1=13	13 × 13 × 256
F6	Dense	512
F7	Dense	256
F8	Dense	10

Layer	Weight formula	Weights	Biases	Total
C1	$7^2\times3\times64$	9,408	64	9,472
P1	0	0	0	0
C2	$5^2\times64\times192$	307,200	192	307,392
P2	0	0	0	0
C3	$3^2\times192\times384$	663,552	384	663,936
C4	$3^2\times384\times256$	884,736	256	884,992
C5	$3^2\times256\times256$	589,824	256	590,080
P3	0	0	0	0
F6	$43{,}264\times512$	22,151,168	512	22,151,680
F7	512×256	131,072	256	131,328
F8	256×10	2,560	10	2,570

Flattened dimension = 13×13×256 = 43,264.
Total = 24,741,450 trainable parameters.

The independent Python formula calculation was compared against executed PyTorch Conv2d, MaxPool2d, Flatten and Linear layers, checking every intermediate shape and every layer count. ReLUs do not change shapes or counts. All comparisons agreed exactly; this is the assigned architecture, not the published AlexNet count.

Task 3. Softmax is a mass function

Let $K \geq 1$ be finite and $z \in \mathbb{R}^K$ have finite entries. Set $Z = \Sigma_{j=1}^{K} \exp(z_j)$. Each exponential is strictly positive and finite; hence $0 < Z < \infty$.

$$s_i(z) = \exp(z_i)/Z > 0, \quad i = 1, \ldots, K.$$

$$\Sigma_{i=1}^{K} s_i(z) = (\Sigma_{i=1}^{K} \exp(z_i))/Z = Z/Z = 1.$$

Therefore, for each fixed score vector z, $s(z)$ defines a probability mass function on the finite class set $\{1, \ldots, K\}$. The probability of a set of classes A is $\Sigma_{i \in A} s_i(z)$.

Why it is not a continuous probability density

The normalization above sums over discrete class indices. It does not integrate over a continuous outcome. Although z varies continuously, z is a conditioning score vector, not the outcome whose probabilities are being assigned. Softmax produces one mass for each class; it introduces no real-valued outcome variable t and no density $f(t)$ satisfying $\int f(t)dt = 1$.

A mass function can be called a density relative to counting measure in measure-theoretic terminology. Here “density” means the continuous-outcome density defined in the question; the distinction is between summation over classes and integration over continuous outcomes.

The numerical check uses max-subtraction for stability and verifies positivity and normalization on finite moderate scores. Strict positivity is an exact real-arithmetic property; floating-point exponentials can underflow to zero for extreme score differences.

Task 4. Attention and permutations

Rows of X are token vectors. $Q=XW_Q$, $K=XW_K$, $V=XW_V$. Let $P_{i,\pi(i)}=1$. Attention is unmasked; softmax acts row-wise.

(a) Scores follow token identities

$$(QK^T)_{ij} = \Sigma_r Q_{ir}K_{jr} = q_i^T k_j.$$

Since $Q'=(PX)W_Q=PQ$ and $K'=PK$, their i-th and j-th rows are $q_{\pi(i)}$ and $k_{\pi(j)}$. Thus:

$$(q'_i)^T k'_j = q_{\pi(i)}^T k_{\pi(j)}.$$

When a token moves from seat 1 to seat 3, every dot product of that token with the other tokens retains its value; only the row and column seats occupied by those values change.

For u=(1,0), v=(0,1), $W_Q=W_K=I$, the score table in seat order (u,v) is [[1,0],[0,1]]. After swapping to (v,u), it is also [[1,0],[0,1]]. The u·u entry moves from (1,1) to (2,2), v·v moves conversely, and u·v and v·u exchange (1,2) and (2,1). Equality of the displayed matrices is due to this symmetric example.

(b) Equivariance of the whole layer

$Q'=PQ$, $K'=PK$, $V'=PV$.

$S'=Q'K'^T/\sqrt{d_k}=P\,S\,P^T$.

For $A=\mathrm{softmax}(S)$, entry $A_{ij}=\exp(S_{ij})/\Sigma_l \exp(S_{il})$. Reindexing l by a permutation leaves the denominator unchanged, so $\mathrm{softmax}(PSP^T)=PAP^T$. Consequently:

$$\mathrm{Att}(PX)=(PAP^T)(PV)=PA(P^TP)V=PAV=P\,\mathrm{Att}(X).$$

The output row associated with each token moves with that token. With $W_V=I$ and the scaling omitted only for this numerical example, let $a=e/(e+1)=0.73105858$ and $b=1/(e+1)=0.26894142$:

$\mathrm{Att}(X)=[[a,b],[b,a]]$,

$\mathrm{Att}(PX)=[[b,a],[a,b]]=P\,\mathrm{Att}(X)$.

(c) Position must affect the dot-product inputs

$PX+E$ has rows $(x_2+e_1,\ x_1+e_2)$.

$P(X+E)$ has rows $(x_2+e_2,\ x_1+e_1)$.

Their difference is $E-PE$, with rows (e_1-e_2, e_2-e_1), which is nonzero if $e_1\neq e_2$. Thus $PX+E\neq P(X+E)$. The seat-specific score is $(x_i+e_i)^T(x_j+e_j)$, so seat information is already present before mixing values. Shared row-wise layers commute with P; composing them with unmasked attention preserves $F(PX)=PF(X)$ absent positional information. The input inequality does not imply every choice of weights produces distinct outputs.

Task 5. Silverbox neural acceleration

The official Silverbox benchmark MAT file contains V1 and V2, each with shape (1,131072), plus overload indicators. V1 is the applied voltage u and V2 the measured output y. The exact slice [42650:46746] contains 4096 samples. Each channel is centered by its own mean over that whole excerpt, as the question requires. Training uses samples 0–3071 and testing 3072–4095 of the excerpt.

Data source: https://www.nonlinearbenchmark.org/benchmarks/silverbox. The official nonlinear-benchmarks loader downloads the linked archive. The downloaded file's SHA-256 is recorded for reproducibility, not asserted against an unavailable publisher checksum:

5c7413a52255af0cb4e2f93fb6903beb
92b08965ba1e6ad0a6ffedfed8477ed4

$$f_s = 610.35 \text{ Hz}, \Delta t = 1/f_s \approx 0.00163840419 \text{ s}.$$
$$\ddot{y} + a\dot{y} + by + cy^3 = gu(t).$$
$$\dot{y} = v,\ \dot{v} = \alpha_\theta(y, v, u).$$

The physical terms represent damping, linear stiffness, cubic stiffness and input gain. They motivate an acceleration function; the network weights are not the four physical coefficients. The implemented model learns acceleration directly, rather than fitting an unrelated static mapping or using a physics residual as a replacement for integration.

For the kept model, normalized inputs are (y/0.1, v/45, u/0.1), in volts, volts/second and volts respectively. The fully connected map is 3→32→32→1 with tanh after each hidden affine layer and a linear output multiplied by 20,000 V/s². Its trainable count is 1,217. Initialization is PyTorch Linear's seeded default. The fixed scale buffers are saved with the weights and used identically during inference.

Differentiable integration and sample alignment

Use classical RK4 on s=(y,v), with $F(s,u) = (v, \alpha_\theta(s,u))$. At a step from sample i to i+1, u at the first and fourth stages is u_i and u_{i+1}; both middle stages use $(u_i + u_{i+1})/2$, i.e. piecewise linear input interpolation.

$$k_1 = F(s_i, u_i),$$
$$k_2 = F(s_i + \Delta t\, k_1/2, (u_i + u_{i+1})/2),$$
$$k_3 = F(s_i + \Delta t\, k_2/2, (u_i + u_{i+1})/2),$$
$$k_4 = F(s_i + \Delta t\, k_3, u_{i+1}),$$
$$s_{i+1} = s_i + \Delta t(k_1 + 2k_2 + 2k_3 + k_4)/6.$$

The first predicted output is the initial position; N samples require N−1 integration steps. Final evaluation initializes $s_0 = (y_0, (y_1 - y_0)/\Delta t)$, and then makes one 4096-point rollout. The state is carried across index 3072 without a reset. The prediction function receives only the input record and the first two measured positions. Measured test outputs are used only afterwards to calculate errors and plots. Whole-excerpt centering is the assignment's prescribed preprocessing.

Task 5. Training and measured results

Only training observations initialize training windows. Training velocity and acceleration estimates come from a Savitzky–Golay filter (9 samples, degree 5, first/second derivatives, Δt). These estimates warm up the acceleration network for 1,200 optimizer steps, excluding five points at each edge. Then optimize multistep rollout loss mean[((ŷ−y)/0.1)²], with 32 independently sampled windows per batch. Training windows avoid the filtered edges; none includes test samples. AdamW uses PyTorch's default weight decay of 0.01; Adam uses zero weight decay.

All runs use seed 42, float32 and one CPU thread. Multistep updates clip the gradient norm at 10 and check finite loss/gradients. The audited implementation also checks finite warm-up loss/gradients without clipping their norm. The window horizon includes the initial sample. Both architectures were compared under multiple rates, horizons, optimizers and update counts. Only completed runs enter the table.

Widths	Optimizer / rate	Updates / horizon	Train RMSE (V)	Test RMSE (V)	Time (s)
[16, 16]	Adam / 0.001	600 / 24	0.004464	0.012823	10.0
[16, 16]	Adam / 0.0003	600 / 48	0.012920	0.025966	17.6
[32, 32]	Adam / 0.001	600 / 32	0.002332	0.008616	12.5
[32, 32]	Adam / 0.0003	600 / 64	0.011109	0.026519	23.3
[16, 16]	AdamW / 0.0005	900 / 48	0.008872	0.021901	25.9
[32, 32]	AdamW / 0.0005	900 / 48	0.002854	0.011710	26.5

Kept run: **w32_lr0.001_h32**. Optimizer Adam, rate 0.001, 600 multistep updates after 1200 warm-up updates, horizon 32, batch size 32. Selection uses measured test RMSE as requested; this score is not an untouched independent estimate. A fresh same-seed training run reproduced every weight tensor and metric exactly. An independent RK4 replay also reproduced all six saved prediction arrays and metrics exactly.

Full-free-run train RMSE = 0.002332454 V.
Full-free-run test RMSE = 0.008616101 V.
Zero-prediction test baseline = 0.060491854 V.

The baseline agrees with the approximately 0.06 V stated in the problem. RMSE is sqrt(mean((prediction−measurement)²)), separately over the first 3072 and final 1024 predictions of the same run. The best network improves substantially on zero, with a larger error on testing than training. No additional coefficient recovery or unseen-record performance is claimed.

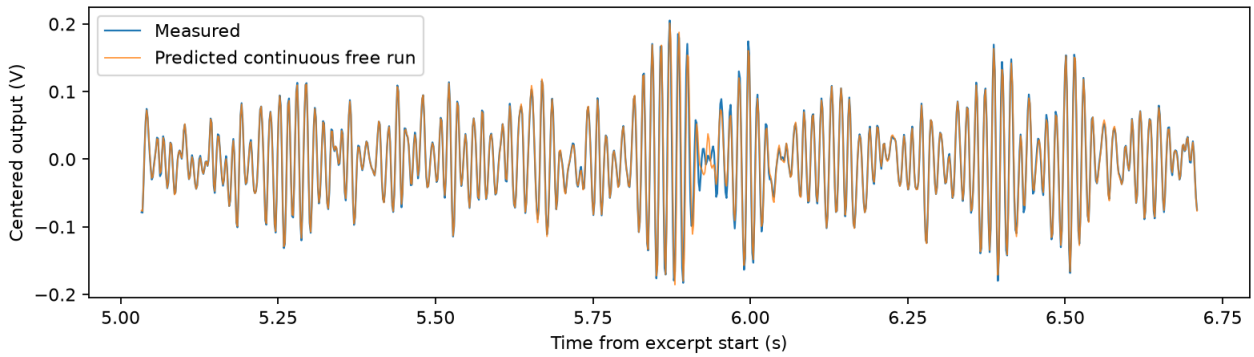


Figure: measured and predicted centered output for the complete test segment. Time is relative to the excerpt start; voltage units are V. This prediction is the uninterrupted free run from sample zero. Saved JSON records contain the measured metrics; the final checkpoint and evaluation command reproduce the prediction and plot.

Task 6. Agentic system and validation status

The implementation uses LangGraph StateGraph with exactly four role nodes: orchestrator, scientist, general and executor. START enters the orchestrator; conditional edges choose a specialist or END; specialist edges return to the orchestrator. State contains chronological {role,text} dialogue, decisions, execution evidence and status. Every role receives prior dialogue, including the latest scientist plan.

Role	Responsibility / access
orchestrator	Real LLM structured decision: next_role, instruction, reason. Routes only; no coding tool. At most eight decisions.
scientist	Duffing equation, neural architecture, state formulation, training, RK4, continuous free-run evaluation and numbered implementation plan. No coding tool.
general	Supporting conceptual questions; no fitting plan or coding tool.
executor	Generates source implementing the latest scientist plan. Only role with the tool. Writes and runs Python; failures and repairs remain in dialogue.

The decision schema rejects invalid roles and missing/empty fields. A generated textual RMSE is insufficient: the executor must save a fresh compatible neural checkpoint, and a separately executed trusted script must print a real full-free-run metric. Successful verification is stored in graph state; only then may finish succeed. A premature finish or exhausted decision limit gives an unsuccessful status.

The tool creates a run-specific directory, writes solution.py, and launches Python with subprocess without a shell. It captures real exit status, stdout and stderr, kills the process group on timeout, and returns actual tracebacks. The whole call, including the independent evaluator, has a 300-second budget. Credentials are stripped from the child environment. Path auditing restricts reads and writes, blocks subprocesses and sockets, and Linux limits constrain CPU, address space and output file size. Python auditing is a practical safeguard, not secure isolation against hostile native code; a container or VM is required for that use.

Actual Task 5 demonstration attempt

Saved trace status: **blocked**. OpenAIRateLimitError("Error code: 429 - {'error': {'message': 'You have no credits remaining. Add credits to continue using the API at https://platform.openai.com/settings/organization/billing/.', 'type': 'insufficient_quota', 'param': None, 'code': 'credit_balance_exhausted'}}").

Real completed model calls: 0. Orchestrator decisions: 0. Role transitions: none completed.

The actual Task 5 request was supplied to task6.run, but no successful end-to-end result is claimed. The saved error above identifies the actual blocker; configured credentials alone do not guarantee API credit or execution success. The Task 5 metrics elsewhere in this report come from direct experiments, not from the agent workflow.

Infrastructure tests are separate evidence

Mock role outputs test all graph routes, chronological role attribution, the eight-decision limit, rejection of finish without execution, and rejection of execution without a scientist plan. Real small subprocess tests verify source writing, stdout/stderr, errors, timeout and out-of-directory write rejection. Another test creates an untrained zero-weight checkpoint and actually runs the trusted evaluator. Its metric is a unit-test artifact, not a fitted Silverbox agent result. The complete real graph demonstration remains required.

Reproduction and remaining submission requirements

Use Python 3.13.9 on Linux; main dependency versions are pinned in requirements.txt. CPU training is sufficient. From the repository root, create/activate .venv and install requirements.txt with the PyTorch CPU index. The README gives complete installation details.

python -m task5.data

python -m task1.verify_networks

python -m task2.parameter_count

python -m task3.verify_softmax

python -m task4.verify_attention

python -m task5.experiments

python -m task5.evaluate results/w32_lr0.001_h32.pt

python -m pytest -q

python -m task6.run

python -m report.build

The experiment sweep reuses a completed run only if its checkpoint exists, its configuration matches, and reevaluated metrics match the JSON record. Invalid or missing artifacts trigger retraining, with old records retained in history. The selected checkpoint is included in Git; data and other checkpoints are excluded. Rerun acquisition and the experiment sweep to reproduce all comparisons. The final technical audit is recorded in report/verification.md.

GitHub repository: publication blocked. No usable repository metadata, authenticated GitHub account or destination URL was available. No push was attempted and no URL is invented. Tasks 5 and 6 must be published and their verified repository URL added to this PDF before final submission.

Task 6: a valid locally configured API key, available API credits and network access are required. If the saved run is unsuccessful, resolve its recorded error and rerun task6.run. Unit tests do not replace the real workflow requirement.

AI assistance disclosure: AI assistance generated and checked the mathematical solutions, implementation, experiments, tests and report. This document makes no claim that the student independently performed those actions. The student should review the work and follow the lecturer’s applicable disclosure rules. No additional lecturer AI-use policy was present in the provided seven-page assignment.

The original assignment requests one solution PDF, code on GitHub for Tasks 5 and 6, and email submission. This package provides the PDF locally but does not claim publication or email delivery.

Sources

Assignment: homework-1.pdf, seven pages, supplied by the user.
Silverbox: nonlinearbenchmark.org/benchmarks/silverbox.
Wigren and Schoukens (2013), Three free data sets for development and benchmarking in nonlinear system identification, ECC, pp. 2933–2938.
LangGraph Graph API: docs.langchain.com/oss/python/langgraph/graph-api.
LangChain structured output: docs.langchain.com/oss/python/langchain/structured-output.